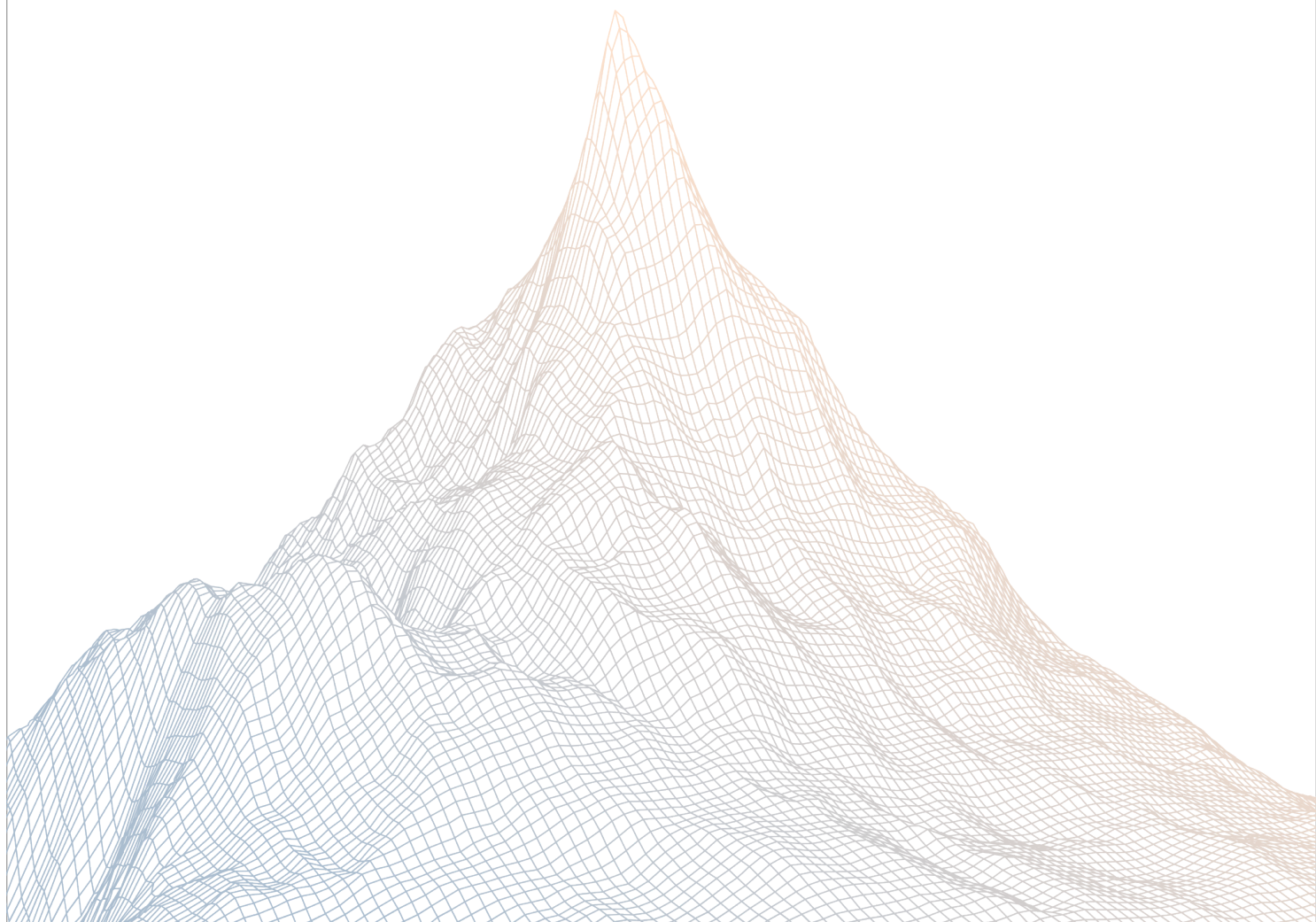


Li.Fi

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

November 12th to November 14th, 2025

AUDITED BY:

Riley Holterhus
kaden

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Li.Fi	4
2.2	Scope	4
2.3	Audit Timeline	5
2.4	Issues Found	5
3	Findings Summary	5
4	Findings	6
4.1	Medium Risk	7
4.2	Low Risk	9
4.3	Informational	15

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Li.Fi

The ultimate cross-chain liquidity aggregator, aggregating cross-chain liquidity networks to DEXs, calculating you the best cross-chain swaps. The future is cross-chain and we make sure you don't have to care. #DeFi

2.2 Scope

The engagement involved a review of the following targets:

Target	catapultar
Repository	https://github.com/lifinance/catapultar
Commit Hash	19e16e4e21a1b07df4e69e1955e2231f30ebf91b
Files	src/**/*.sol

2.3 Audit Timeline

November 12, 2025	Audit start
November 14, 2025	Audit end
November 19, 2025	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	3
Informational	5
Total Issues	9

3

Findings Summary

ID	Description	Status
M-1	Batches can be submitted in any order	Acknowledged
L-1	Signers have no control over gas limits	Acknowledged
L-2	ERC-1271 signatures can be replayed across networks	Acknowledged
L-3	Only partial nonce included in CallReverted event	Acknowledged
I-1	Self-call upgrade considerations	Acknowledged
I-2	Calls struct has different order than CALLS_TYPE_HASH	Resolved
I-3	Incorrect padded length comment	Resolved
I-4	Incorrect documented byte range	Resolved
I-5	Extra cloning strategy documented	Resolved

4

Findings

4.1 Medium Risk

A total of 1 medium risk findings were identified.

[M-1] Batches can be submitted in any order

SEVERITY: Medium

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [Catapultar.sol#L212](#)

Description:

The Catapultar contract uses unordered nonces, which means that any batch with a valid signature can be submitted independently and in any order relative to other batches.

For example, consider a batch of batches submitted using ERC-7821 mode `0x010000000000078210002`. This mode is designed to include multiple independently authorized batches within a single outer array. Each batch has its own signature, which authorizes it individually. Anyone observing these signatures in the mempool can reorder, add, or remove batches from the outer array, or submit them directly without using the batch of batches structure at all. More generally, whenever there is more than one valid signature in the mempool, the corresponding batches can be executed in any order.

The only place where execution order is guaranteed is within an individual batch. This also applies to batches that recursively call `execute()`, since the `msg.sender = address(this)` shortcut in `_validateOpData()` means that inner batches don't need signatures. As a result, it is still possible to construct more complex calls with a specific order.

Recommendations:

Consider the implications of this behavior. If signers understand that individual batches can always be reordered, the current design may be acceptable and recursive `execute()` calls may be the preferred pattern for complex flows. However, if this behavior is too subtle or error-prone, consider modifying the logic to require sequential nonces.

Li.Fi: Acknowledged. Batch of Batches does not lie in a main execution path. Additionally, for conditional order execution, a traditional batch should be used for the best execution

guarantees.

We have reviewed the ERC-7821 and with the help of the auditors examined how alternative account implement similar functionality. Based on these findings, we will not implement a fix for this issue. Sequential nonces does not fix DoS issues or replaying transactions.

We have considered requiring the executor to be part of the signed `calls` struct but that is not scoped for this version.

4.2 Low Risk

A total of 3 low risk findings were identified.

[L-1] Signers have no control over gas limits

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [ERC7821LIFI.sol](#)

Description:

The ERC7821 batch format includes only a target, value, and calldata for each call. It does not allow the signer to specify the amount of gas that should be forwarded to each call. Because anyone can ultimately submit the `execute()` call on-chain (by observing signatures in the mempool), this opens up the possibility for a submitter to manipulate the gas available to a subcall in order to alter its outcome.

For example, consider a contract with the following logic:

```
contract ExampleContract {
    function exampleFunction1() {
        try this.exampleFunction2() {
            // ...
        } catch {
            // ...
        }
    }
}
```

A malicious submitter could deliberately forward a smaller amount of gas, which could trigger the catch block, even though the signer observed an overall success when they simulated the call themselves.

Recommendations:

Consider the implications of this behavior. If signers are meant to control gas usage, consider adding a `gasLimit` field so they can specify how much gas should be forwarded per call.

Li.Fi: Acknowledged. We are not interacting with any contracts where this poses an issue. It should also be noted that unless a contract is a particularly big gas guzzler, the left over gas might not be enough to emit the `CallReverted` event which will always be emitted before either continuing or reverting.

[L-2] ERC-1271 signatures can be replayed across networks

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [Catapultar.sol#L121](#)

Description:

In `Catapultar.isValidSignature`, we include a rehashing scheme which includes `address(this)` to ensure that signatures signed for use by this contract can only be used by this contract address:

```
function isValidSignature(
    bytes32 hash,
    bytes calldata signature
) public view virtual returns (bytes4 result) {
    bool success = SignatureCheckerLib.isValidSignatureNowCalldata(
        owner(),
        EfficientHashLib.hash(
            REPLAY_PROTECTION, // Offset hash to ensure no standard payload
                                replicates this structure.
            asUnsafeBytes32(address(this)),
            hash
        ),
        signature
    );
    assembly ("memory-safe") {
        // `success ? bytes4(keccak256("isValidSignature(bytes32,bytes)))` :
        // 0xffffffff`.
        // We use `0xffffffff` for invalid, in convention with the reference
        // implementation.
        result := shl(224, or(0x1626ba7e, sub(0, iszero(success))))
    }
}
```

However, while the rehashing scheme used does prevent replay on contracts with a different address, it does not protect against replay on the same contract address across

different networks. In case the same owner has Catapultar contracts with the same address on different chains, ERC-1271 signatures used on one network could be replayed on another network, causing unexpected effects.

Recommendations:

Implement a stronger defensive rehashing scheme, e.g. [ERC-7739](#), or at least include the `chainId` in the current rehashing scheme.

Li.Fi: Acknowledged. The functionality is intended to replicate ECDSA signatures which can be replayed.

[L-3] Only partial nonce included in CallReverted event

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: High

Target

- [ERC7821LIFI.sol#L93-L97](#)

Description:

In ERC7821LIFI, during crafting of the extraData, we retrieve and include only the least significant 23 bytes of the nonce used for the batch:

```
// Add the last 23 bytes of the first words of opData to extraData
assembly ("memory-safe") {
    let word := calldataload(opData.offset)
    extraData := or(extraData, shr(8, shl(mul(9, 8), word)))
}
```

However, when validating and using the nonce, we use the full 32 byte word:

```
uint256 nonce;
assembly ("memory-safe") {
    nonce := calldataload(opData.offset)
}
_useUnorderedNonce(nonce);
```

```
function _useUnorderedNonce(
    uint256 nonce
) internal {
    if (nonce == 0) revert InvalidNonce();
    (uint256 wordPos, uint256 bitPos) = bitmapPositions(nonce);
    uint256 bit = 1 << bitPos;
    uint256 flipped = nonceBitmap[wordPos] ^= bit;

    if (flipped & bit == 0) revert InvalidNonce();
}
```

As a result, we ignore the upper 9 bytes of the nonce in the `extraData`, resulting in the `CallReverted` event not including the full nonce used.

Recommendations:

Consider either including the full 32 byte nonce in the `extraData`, or limit nonces to being ≤ 23 bytes so that they can fully fit in `extraData` as it is.

Li.Fi: Acknowledged. We do not intent to fix as it is not possible for the nonce to exposed in unpredictable ways.

4.3 Informational

A total of 5 informational findings were identified.

[I-1] Self-call upgrade considerations

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [Catapultar.sol#L184-L192](#)

Description:

Catapultar contracts deployed as ERC-1967 proxies can be upgraded either directly by the owner or via a self-call. When an upgrade is performed through a self-call during a batch, the execution context of subsequent calls in the batch is complex and should be carefully considered.

Consider a batch that includes a self-call to `upgradeToAndCall()` in the middle. All earlier calls in the batch (including any self-calls) will be using the pre-upgrade code. After the upgrade is performed, the loop that iterates through the batch is still the pre-upgrade code (i.e. even if the upgraded implementation has different `_execute()` behavior, the old behavior is still used at this point). However, any self-calls that occur later in the batch would reach the post-upgrade code.

In other words, if a batch includes an upgrade transaction, the remaining external calls in the batch will continue as usual under the original implementation, but any self-calls/callbacks after the upgrade will be affected by the new implementation.

Recommendations:

Keep this behavior in mind. It might be simplest to always perform upgrades in singleton batches or at the end of a batch.

Li.Fi: Acknowledged.

[I-2] calls struct has different order than CALLS_TYPE_HASH

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [LibCalls.sol#L7-L20](#)

Description:

The calls struct is currently defined as:

```
struct Calls {
    bytes32 mode;
    uint256 nonce;
    ERC7821.Call[] calls;
}
```

The corresponding CALLS_TYPE_HASH is defined as:

```
bytes32 constant CALLS_TYPE_HASH =
    keccak256(bytes("Calls(uint256 nonce,bytes32 mode,Call[]
calls)Call(address to,uint256 value,bytes data)"));
```

The order of nonce and mode differs between the two definitions. While this does not cause any specific issues, it may lead to confusion in the future.

Recommendations:

Consider reordering the calls struct to match the CALLS_TYPE_HASH definition.

Li.Fi: Resolved with [PR-7](#).

Zenith: Verified.

[I-3] Incorrect padded length comment

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [ERC7821LIFI.sol#L151](#)

Description:

In ERC7821LIFI._execute, we compute the sizeAfterPad of the returndata, rounding up to the nearest multiple of 32 for ABI alignment:

```
// Compute the padded length for ABI alignment. (rdsize + rdsize % 32)
let sizeAfterPad := and(add(returndatasize(), 31), not(31))
```

While the computed sizeAfterPad value is correct, the comment is incorrect as `rdsize + rdsize % 32` will only be correct if `rdsize` happens to be a multiple of 16.

Recommendations:

Adjust the above noted comment to include logic that will correctly result in rounding up to the nearest multiple of 32:

```
// Compute the padded length for ABI alignment. (rdsize + rdsize % 32)
// Compute the padded length for ABI alignment. ((rdsize + 31) / 32 * 32)
let sizeAfterPad := and(add(returndatasize(), 31), not(31))
```

Li.Fi: Resolved with [PR-9](#).

Zenith: Verified.

[I-4] Incorrect documented byte range

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [ERC7821LIFI.sol#L75-L76](#)

Description:

In ERC7821LIFI.sol, we document extraData as follows:

```
// extraData has the following format:  
// [0] revert flag  
// [1 .. 22] Nonce extract  
// [23 .. 31] Index
```

However, while this documents the nonce extract as consuming a 22 byte range, we actually include 23 bytes:

```
// Add the last 23 bytes of the first words of opData to extraData  
assembly ("memory-safe") {  
    let word := calldataload(opData.offset)  
    extraData := or(extraData, shr(8, shl(mul(9, 8), word)))  
}
```

Recommendations:

Adjust the documented byte range for both the nonce extract and the index:

```
// extraData has the following format:  
// [0] revert flag  
// [1 .. 22] Nonce extract  
// [1 .. 23] Nonce extract  
// [23 .. 31] Index  
// [24 .. 31] Index
```

Li.Fi: Resolved with [PR-8](#).

Zenith: Verified.

[I-5] Extra cloning strategy documented

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [CatapultarFactory.sol#L14-L17](#)
- [Catapultar.sol#L43](#)

Description:

In `CatapultarFactory.sol`, we document three cloning strategies being supported:

- * Three cloning strategies are supported:
- * - Non-upgradeable minimal `PUSH_0` clone **for** a low cost batch execution account.
- * - Non-upgradeable embedded args clone **for** a short lived account with a pre-configured allowed call.
- * - Upgradeable ERC1967 proxy **for** a durable long term account.

However, only the non-upgradeable minimal `PUSH_0` and upgradeable ERC1967 proxy strategies are supported.

Similarly, in `Catapultar.sol`, we note that the contract can be used via 3 cloning strategies:

- * The **contract** **is** intended to be used via **3** cloning strategies:

Recommendations:

Adjust the above listed comments to only include the two supported cloning strategies.

Li.Fi: Resolved with [PR-6](#).

Zenith: Verified.